Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

SINATRA PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on routes, middleware, templates, helpers, and testing

TOTAL: 10 QUESTIONS

Q1 What is Sinatra and how does it differ from Ruby on Rails? — Threat Model

Q2 How do you define and read URL parameters in Sinatra? — Architecture

Q3 How does Sinatra handle request data (body, headers, forms)? — Configuration

Q4 What template engines does Sinatra support and how do you render them? — Hardening

Q5 How do Sinatra before/after filters work? — SSO / IdP

Q6 How do you define and use helper methods in Sinatra? — Observability

Q7 How do you handle errors and define custom error pages? — Lifecycle

Q8 What is the difference between classic-style and modular Sinatra? — Compliance

Q9 How do you manage sessions and cookies in Sinatra? — Testing

Q10 How do you test a Sinatra application with Rack::Test? — Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Sinatra is a DSL for building

Mitigation

Sinatra is a DSL for building web apps in Ruby with minimal structure. There is no MVC, no ORM, no asset pipeline by default. You define routes with get/post/put/delete and return a string or render a template. Rails is a full framework; Sinatra is a micro-framework.

A2 get '/users/:id' do; user_id = params[:id];

Primitives

get '/users/:id' do; user_id = params[:id]; end. Wildcard: get '/files/*' do; path = params['splat'].first; end. Query strings are also in params: GET /search?q=ruby gives params[:q] == 'ruby'. Route blocks have access to request, response, and session.

A3 request.body.read returns the raw request body.

Hardening

request.body.read returns the raw request body. request.env['HTTP_AUTHORIZATION'] reads a header. params includes both query string and form fields from application/x-www-form-urlencoded or multipart. For JSON: JSON.parse(request.body.read).

A4 Sinatra supports ERB (erb :view), Haml

Gotchas

Sinatra supports ERB (erb :view), Haml (haml :view), Slim, Mustache, and others. Install the corresponding gem, then call the helper with a symbol matching the file name in views/. Pass local variables: erb :user, locals: { user: @user }.

A5 before do; @user = User.find(session[:user_id]);

Federation

before do; @user = User.find(session[:user_id]); end runs before every route. after do; log_response(response); end runs after. Filters accept a pattern: before '/admin/*' do; halt 401 unless @user&.admin?; end scopes to matching paths.

A6 Helpers inside a helpers do block

Observability

Helpers inside a helpers do block or a module included via helpers MyHelpers become available in both route blocks and templates. def link_to(text, href); "#{h(text)}"; end. h() is the built-in HTML escape helper.

A7 not_found do; 'Page not found'; end

Automation

not_found do; 'Page not found'; end handles 404. error 500 do; 'Server error'; end handles 500. error MyCustomException do; status 400; 'Bad input'; end catches a specific exception class. error do catches any unhandled exception.

A8 Classic style (require 'sinatra') defines routes

Governance

Classic style (require 'sinatra') defines routes at the top level simple but pollutes the global namespace. Modular style (class MyApp < Sinatra::Base) is a reusable class you mount in a Rack config.ru. Modular is preferred for tests and embedding in larger apps.

A9 Enable sessions with enable :sessions (uses

Assessment

Enable sessions with enable :sessions (uses Rack::Session::Cookie with a random secret by default). Set a secret with set :session_secret, 'long-random-string'. session[:user_id] = user.id stores data. Cookies: response.set_cookie('name', value: 'x', httponly: true).

A10 Include Rack::Test::Methods and define app {

Optimization

Include Rack::Test::Methods and define app { Sinatra::Application } (or MyApp). Call get '/path', headers: {'Authorization'=>'Bearer token'}. Assert last_response.status == 200 and last_response.body. set :environment, :test to suppress output.